

Dokumentation Übung 2

1. Einführung

Aufgrund verschiedenster Probleme der ursprünglichen Abgabe wurde für diese Abgabe ein neues Labyrinth erstellt. Lediglich für die Reflektionen hat die Zeit nicht gereicht um diese erneut durch verschiedenste Reflection Probes darzustellen. Im Rahmen dieser Übung wurde eine Unity-Anwendung erstellt, in der ein NPC (nicht-spielbarer Charakter) mithilfe einer externen KI dynamisch auf das Verhalten eines Spielers reagiert. Der Fokus lag auf der Kombination moderner KI-Modelle mit klassischem Game-Design. Dabei wurde eine REST-API für KI-Kommunikation entwickelt und ein NPC samt Bewegung, Logik und Animationen aufgebaut. Besonderes Augenmerk lag auf der Korrektheit der Navigation, Kollisionen, visueller Darstellung sowie der systematischen Abfrage und Interpretation von Spielerdaten in Echtzeit.

2. NPC-Erstellung, Rigging und Animationen

Ein wesentlicher Teil des Projekts war der Import eines NPC-Modells. Der NPC wurde aus einem humanoiden FBX-Modell erstellt und mit einem Animator-Controller ausgestattet. Zunächst wurde ein Charakter ohne korrektes Rig geladen, was zu verschiedensten Fehlern führte bezüglich eines nicht zutreffenden Skeletts. Erst durch den Import eines neuen Modells mit vollständigem Humanoid-Rig konnten die Animationen erfolgreich auf den Charakter gemappt werden. Die Animator-Komponente wurde daraufhin ergänzt. Die Navigation erfolgt über einen NavMeshAgent, der das Ziel verändert, z.B. beim weglaufen.

Folgende Animationen wurden genutzt und im Animator-Controller miteinander verknüpft:

- Idle: Standardzustand, wenn keine Interaktion erfolgt, NPC steht in Gegend herum und atmet
- Walking: NPC bewegt sich langsam
- Running: NPC bewegt sich schnell
- CrawlBackwards: NPC kriecht rückwärts auf dem Boden
- Waving: NPC winkt
- RunAway: NPC bewegt sich weg von Spieler, läuft dabei
- Sad Idle: NPC steht in Gegend herum und schaut traurig
- Happy Idle: NPC steht in Gegend herum und schaut glücklich
- Crouched: NPC hockt sich auf Boden, besteht aus 3 zusammengesetzten Animationen
 - StandingToCrouched
 - Crouching Idle
 - CrouchToStand

Diese wurden über Trigger-Parameter aktiviert. Die Übergänge wurden ohne `Exit Time` gestaltet, um sofortige Reaktion zu gewährleisten. Über das Skript `NPCMovement.cs` wird je nach KI-Ausgabe der passende Trigger gesetzt.

Dazu gehören:

- Speed (Int), verwendet für Unterschied zwischen Breathing Idle, Walking und Running
- Trigger:
 - Idle
 - Wave
 - RunAway
 - CrawlBackwards
 - Crouch
 - Sad
 - Happy

3. KI-Anbindung über REST-API

Die Sprach-KI wurde lokal über LM Studio gehostet, erreichbar über VPN auf dem Port 1234. Ein Flask-Server („api_server.py“) wurde eingerichtet, um als Vermittler zwischen Unity und dem KI-Backend zu fungieren. Unity sendet via „UnityWebRequest“ POST-Anfragen an `http://localhost:5050/chat`, welche durch das Flask-Interface an LM Studio weitergeleitet werden. Dort läuft ein Sprachmodell – in diesem Fall das Modell „deepseek-coder-v2-lite-instruct“. Die Daten wurden im OpenAI-kompatiblen Format als JSON gesendet, einschließlich „model“, „messages“, „temperature“.

Die Anfragen enthalten aktuelle Informationen über die Spielerposition und Geschwindigkeit.

Ein typisches JSON-Format sieht folgendermaßen aus:

```
{
  "model": "deepseek-coder-v2-lite-instruct",
  "messages": [{"role": "user", "content": "Spieler 2.3m entfernt, Geschwindigkeit: 4.6 m/s"}]
}
```

Auf Unity-Seite wurde dies durch das Skript `NPCStateSender.cs` umgesetzt. Es sendet per „UnityWebRequest.Post()“ regelmäßig die aktuelle Entfernung und Geschwindigkeit des Spielers an die API. Die Antwort der KI wird anschließend geparsed und an „NPCMovement.cs“ übergeben, das dann basierend auf dem String („run_away“, „wave“, etc.) den passenden Trigger im Animator auslöst und bei Bedarf den Agenten neu ausrichtet. Des Öfteren wurde dabei eine Fehlermeldung ausgegeben, da die Verbindung zur KI nicht funktioniert hatte, dies wurde allerdings nach einigen Versuchen behoben.

4. Verwendete Unity-Skripte

Folgende Skripte wurden für die NPC-Interaktion und KI-Anbindung verwendet:

- **NPCMovement.cs:**
Dieses Skript steuert die Navigation des NPCs über NavMeshAgent, reagiert auf Trigger aus der KI und initiiert periodische Statusabfragen. Distanzen und Geschwindigkeiten zum Spieler werden darin berechnet, interpretiert und regelmäßig an die „NPCStateSender“-Komponente weitergeleitet.
- **NPCStateSender.cs:**
Verantwortlich für die API-Kommunikation. Sendet die Nachricht an das Flask-Backend, empfängt die KI-Antwort und ruft „ReactToMessage()“ in „NPCMovement“ auf. Fehlerbehandlung und Timeout-Überwachung sind enthalten. Die Methode „SendMessageToAPI“ wird regelmäßig vom Movement-Skript aufgerufen.
- **api_server.py:**
Stellt ein einfaches REST-API mittels Flask bereit. Führt Protokoll über Konversationen („message_history“) und sendet aktuelle Prompts an LM Studio. Konvertiert die interne Form in OpenAI-kompatibles JSON, welches von LM Studio verarbeitet werden kann.

5. Automatisierung und Periodisches Verhalten

Zu Beginn wurde der KI-Aufruf manuell über die Taste T ausgelöst um die Verfügbarkeit zu testen. Dies wurde ersetzt durch ein periodisches System im „NPCMovement“-Skript, welches alle 2 Sekunden prüft, wie weit der Spieler entfernt ist und wie schnell er sich bewegt. Bei zu hoher Geschwindigkeit in kurzer Distanz soll der NPC fliehen, bei langsamer Annäherung freundlich reagieren. Allerdings wurden Anfragen an die KI aufgrund von Performance lediglich alle 7 Sekunden gesendet, wobei darauf geachtet wurde, dass keine Anfrage gesendet wird, wenn zu der letzten Anfrage noch keine Antwort vorliegt.

Die durchschnittliche Geschwindigkeit des Spielers wird über ein gleitendes Fenster berechnet, um Sprünge oder Messfehler zu vermeiden. Zudem wird ein initialer Prompt zu Beginn übermittelt, um der KI ihre Handlungsoptionen zu erklären. Dieser „System Prompt“ gibt an, dass folgende Reaktionen erlaubt sind: idle, wave, run_away, crouch, crawl_backwards, sad und happy. Zusätzlich wird in „api_server.py“ eine „message_history“ gepflegt, welche die letzten Nachrichten enthält. Dadurch kann die KI sich an frühere Situationen erinnern und Entscheidungen konsistenter treffen. Dies hat die Qualität und Konsistenz der Antworten spürbar verbessert. Vor allem sorgte dies für kürzere und gezielte Antworten. Dies war besonders ein Problem, da anfangs trotz vorherigem Initialem Prompt, die Antworten der KI manchmal nicht zu gebrauchen waren oder sehr lange Texte bei rauskamen. Der letztendlich Prompt lautet wie folgt:

Der Spieler bewegt sich durch ein Labyrinth und kommt dir manchmal näher. Sofern der Spieler schneller als üblich auf dich zuläuft, kannst du Angst verspüren und deine Reaktionen

entsprechend anpassen. Allerdings kannst du auch freundlich gegenüber dem Spieler reagieren, sofern du ihn nicht als Bedrohung wahrnimmst. Du kannst dieselbe Aktion mehrfach hintereinander ausführen, musst es aber nicht. Deine Aufgabe ist es, realistisch zu reagieren. Dir stehen verschiedene Aktionen zur Auswahl, die du jeweils mit einem bestimmten Wort als Antwort zurückgibst. Wenn du nichts tust und stehen bleibst, lautet die Antwort idle. Wenn du winkst, lautet sie wave. Wenn du wegläufst, run away. Wenn du dich duckst, crouch. Wenn du rückwärts krabbelst, crawl backwards. Wenn du traurig herumstehst, sad, und wenn du glücklich herumstehst, happy. Wichtig ist, dass du in allen Antworten ab jetzt nur die genannten Begriffe verwendest und keine anderen Wörter oder Zeichen. Diese Regeln gelten dauerhaft und müssen in allen künftigen Antworten berücksichtigt werden. Als Beispiel: Spieler 3,4 Meter entfernt Geschwindigkeit 0.0 m/s, deine Antwort darauf wäre: happy.

6. Aufgetretene Probleme und Lösungen

Während der Entwicklung traten zahlreiche technische Probleme auf:

- KI reagierte nicht → VPN nicht aktiv oder Server nicht gestartet
- Modellfehler bei Animationen → neues Modell mit vollständigem Rig geladen
- API-Fehler 500 oder Timeout → Modellname falsch oder LM Studio nicht verfügbar, sowie Nutzung falscher IPs
- NPC bewegte sich ohne Animation → Animator-Controller falsch verbunden oder Trigger fehlten
- NPC konnte durch Spieler laufen → fehlender Collider am NPC, IsTrigger deaktiviert
- ...

7. Fazit

Das Projekt hat gezeigt, wie sich klassische Spielelemente mit modernen KI-Technologien verbinden lassen. Die größte Herausforderung bestand darin, die einzelnen Komponenten (3D-Umgebung, NPC, Netzwerk, Sprachmodell) zuverlässig miteinander zu koppeln. Durch systematische Fehlersuche, Testing und ein Verständnis der Kommunikationsstruktur konnte ein NPC geschaffen werden, der mithilfe einer KI auf verschiedenste Situationen reagieren kann. Allerdings kann das Verhalten des NPCs noch stark verbessert werden. Bisher wurde es nicht geschafft, dass der NPC selbstständig das Labyrinth abläuft, indem es seine Umgebung an die KI vermittelt. Es ist lediglich möglich, den NPC mithilfe eines Nav Mesh Agents dem NPC ein Ziel zu geben, zu welchem er sich mithilfe der Unity AI bewegen kann. Die Ergebnisse können allerdings leicht erweitert werden – etwa durch weitere Reaktionsarten, Dialogsysteme oder Integration von Sprachausgabe. Zudem könnten die Animationen und deren Übergänge weiter verbessert werden.